

Robowars 2026 GreenShift

Green Shift Sumo Robot: ALL

1. Bill of Materials (BOM)

The following hardware components are utilized in the robot's architecture.

Qty	Component	Function / Notes
1	Freenove ESP32-S3-WROOM Lite	Main microcontroller and logic core. 16mb
1	ESP32-C3 Mini	Telemetry coprocessor (UART communication).
2	Cytron MD13S	DC Motor Drivers for drive system.
4	JSumo Micro ML2	Edge/line sensors for ring boundary detection.
4	JSumo JS200XF	Digital distance sensors for opponent detection.
1	TM171 IMU	Inertial measurement unit for orientation/heading.
1	MicroStart Sumo	IR start module.
2	ACS724 Hall Effect Sensor (40A)	High-current monitoring for drive motors.
1	TXB0104 Bidirectional Logic	4-Channel logic level converter (3.3V ↔ 5V).
1	Micro BEC (5V/9V Adj, DIY-SE-0509)	Synchronous buck regulator to step down 6S voltage.
1	6S LiPo Battery	Main power source.

2. Safe Hardware Pin Allocation (ESP32-S3)

This pinout utilizes the safe UART layouts and avoids Octal PSRAM/Crystal interference.

Component	Signal	GPIO	Engineering Justification
Battery Monitor	Analog (ADC1)	GPIO 2	Safe ADC1 channel (unaffected by Wi-Fi).
ACS724 Current 1	Analog (ADC1)	GPIO 1	Safe ADC1 channel.
ACS724 Current 2	Analog (ADC1)	GPIO 4	Safe ADC1 channel.
Micro ML2 (x4)	Digital IN	GPIO 5, 6, 7, 10	Safe ADC1 pins used as clean digital inputs.
JS200XF (x4)	Digital IN	GPIO 11, 12, 13, 14	ADC2 pins (reliable for pure digital signals).
TM171 IMU	TX / RX (UART1)	GPIO 17 / 18	Mapped to UART1, leaving UART0 free for debug.
ESP32-C3 Mini	TX / RX (UART2)	GPIO 8 / 9	Mapped to hardware UART2 (repurposed I2C pins).
Cytron MD13S right	PWM / DIR	GPIO 41 / 42	Safe JTAG pins (MTDI/MTMS), 3.3V logic.
Cytron MD13S left	PWM / DIR	GPIO 40 / 39	Safe JTAG pins (MTCK/MTDO), 3.3V logic.
MicroStart Sumo	Digital IN	GPIO 15	Safe RTC GPIO, configured as Hardware ISR.

1 Software Architecture & Workspace Layout

To maintain a deterministic and easily debuggable system, the PlatformIO workspace is structured as an asymmetrical multiprocessing (AMP) architecture. This approach separates heavy I/O operations (Nervous System) from high-level decision-making (Brain) using FreeRTOS tasks pinned to specific ESP32-S3 cores [?].

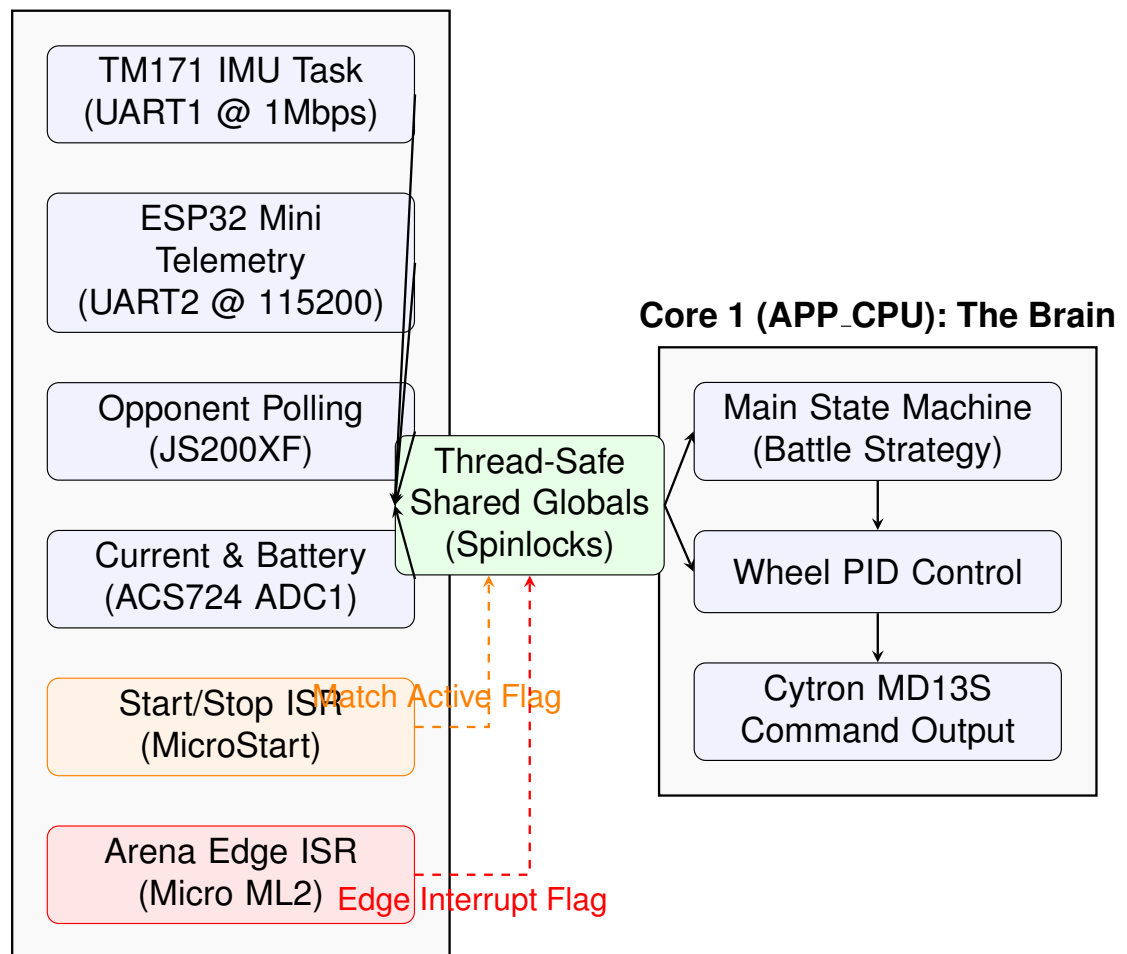
- `lib/` (Third-Party Libraries)
 - `EasyProtocol/`: Logic for 1Mbps UART communication with the TM171 IMU.
- `include/` (Headers)

- `PinConfig.h`: Single source of truth for GPIO assignments, avoiding Octal PSRAM interference.
- `SharedGlobals.h`: Defines the `SharedSensorData` struct and `portMUX_TYPE` spinlocks for thread-safe cross-core communication.
- `src/` (Implementation)
 - **Core 0 (Nervous System):**
 - * `IMU.cpp`: Dedicated task for non-blocking 1Mbps packet parsing.
 - * `Sensors.cpp`: 100Hz task for ADC (Current/Battery) and digital (Opponent) polling.
 - * `Telemetry.cpp`: 20Hz task formatting UART2 packets for the ESP32-C3 coprocessor.
 - **Core 1 (Brain):**
 - * `Strategy.cpp`: Implementation of the Battle State Machine (Classic vs. Matador).
 - * `Motors.cpp`: Dual-channel 20kHz PWM control with PID-based velocity scaling.
 - * `main.cpp`: Entry point that initializes tasks on Core 0 and runs the primary strategy loop on Core 1.
 - **Hardware Reflexes (ISRs):**
 - * `Start.cpp` & `Edge.cpp`: Hardware Interrupt handlers that trigger zero-latency volatile flags for match state and ring boundaries.

2 Dual-Core Processing Strategy

The ESP32-S3 utilizes an asymmetric multiprocessing architecture to prevent I/O bottlenecks from delaying motor responses. The golden rule of this architecture is: **Core 0 gathers the data; Core 1 acts on it.**

Core 0 (PRO_CPU): Nervous System



As shown above, Core 0 isolates the heavy I/O operations. Core 1 is purely dedicated to thinking and moving, completely decoupled from UART buffering or ADC conversion times. The Micro ML2 edge sensors and MicroStart module operate as hardware interrupts, allowing them to bypass the standard polling cycle and immediately signal Core 1 if the robot needs to stop or reverse.

3 Battle Strategy & Sensor Logic

The autonomous battle logic is divided into two distinct operational modes. The active mode is determined at initialization by the boolean flag `executeMatadorProtocol`. This allows the robot to seamlessly switch between a brute-force approach for standard opponents and an evasive, trap-based strategy for advanced competitors.

3.1 Sensor Geometry & Targeting

The hardware utilizes an asymmetrical sensor layout to create a targeted "killzone" just ahead of the blade, while maintaining defensive awareness on the flanks.

3.2 Standard Protocol (`executeMatadorProtocol = false`)

When facing simpler opponents, the state machine utilizes a brute-force targeting system based on the crossed trajectories of the front sensors. Because the outer sensors cross over the center line, the steering logic is inverted to ensure rapid target acquisition:

- **Left Sensor Triggered:** The opponent is located on the right side of the arena. Core 1 commands the Cytron MD13S drivers to pivot right.
- **Right Sensor Triggered:** The opponent is located on the left side of the arena. Core 1 commands a left pivot.
- **Center Sensor Only:** The opponent is directly ahead but outside the optimal focal point. The robot approaches at a controlled medium velocity to prevent overshooting.
- **All Three Triggered:** The opponent is perfectly centered within the designated “Killzone.” Core 1 outputs 100% PWM for an immediate ramming maneuver.

3.3 Matador Protocol (`executeMatadorProtocol = true`)

For highly maneuverable or edge-skirting opponents, the standard head-on charge is bypassed in favor of a bait-and-switch maneuver.

1. **Evasive Opening:** Upon the start signal from the MicroStart module, the robot immediately executes a reverse arc (back-left), breaking the opponent’s initial line of sight.
2. **Decoy Deployment:** A lateral flag, actuated by an STS3215 intelligent bus servo, snaps out on the robot’s left flank. Advanced opponents utilizing wide-angle sensors will detect this protrusion and pivot to attack it.
3. **The Trap:** The left flank acts as intentional bait. As the opponent charges the flag, the robot pivots sharply to the left. This rotational maneuver exposes the opponent’s weaker side profile directly into the crossed focal point of the front sensor array, triggering a full-power ramming sequence.

Note on Defensive Priority: Regardless of the active protocol, the Micro ML2 edge sensors operate as hardware interrupts. If a ring boundary is detected, Core 1 instantly overrides all offensive commands to execute a zero-latency evasion sequence.

4 Finite State Machine (FSM) Implementation

The core decision-making framework of the robot is governed by a priority-based Finite State Machine (FSM). This logic relies on four primary files: `Strategy.h` defines the `RobotState` enumeration and execution prototypes; `Strategy.cpp` houses the primary

`switch` statement that evaluates the current state against the `SharedSensorData` snapshot to determine the next behavior; `Motors.cpp` translates these abstract state decisions into physical 20kHz PWM signals and directional pin logic; and finally, `main.cpp` orchestrates the continuous loop, fetching thread-safe sensor snapshots via FreeRTOS spinlocks and feeding them into the FSM.

The architecture guarantees that safety and testing overrides take precedence over combat logic. As illustrated below, the evaluation sequence explicitly begins by checking the `debug_motor` flag. If `false`, the system proceeds to standard battle execution; if `true`, the FSM is bypassed entirely in favor of a hardware calibration sequence. During combat, hardware interrupts (such as the Micro ML2 edge sensors) instantly trigger the highest-priority `STATE_EVADE`, overriding any active attack vectors.

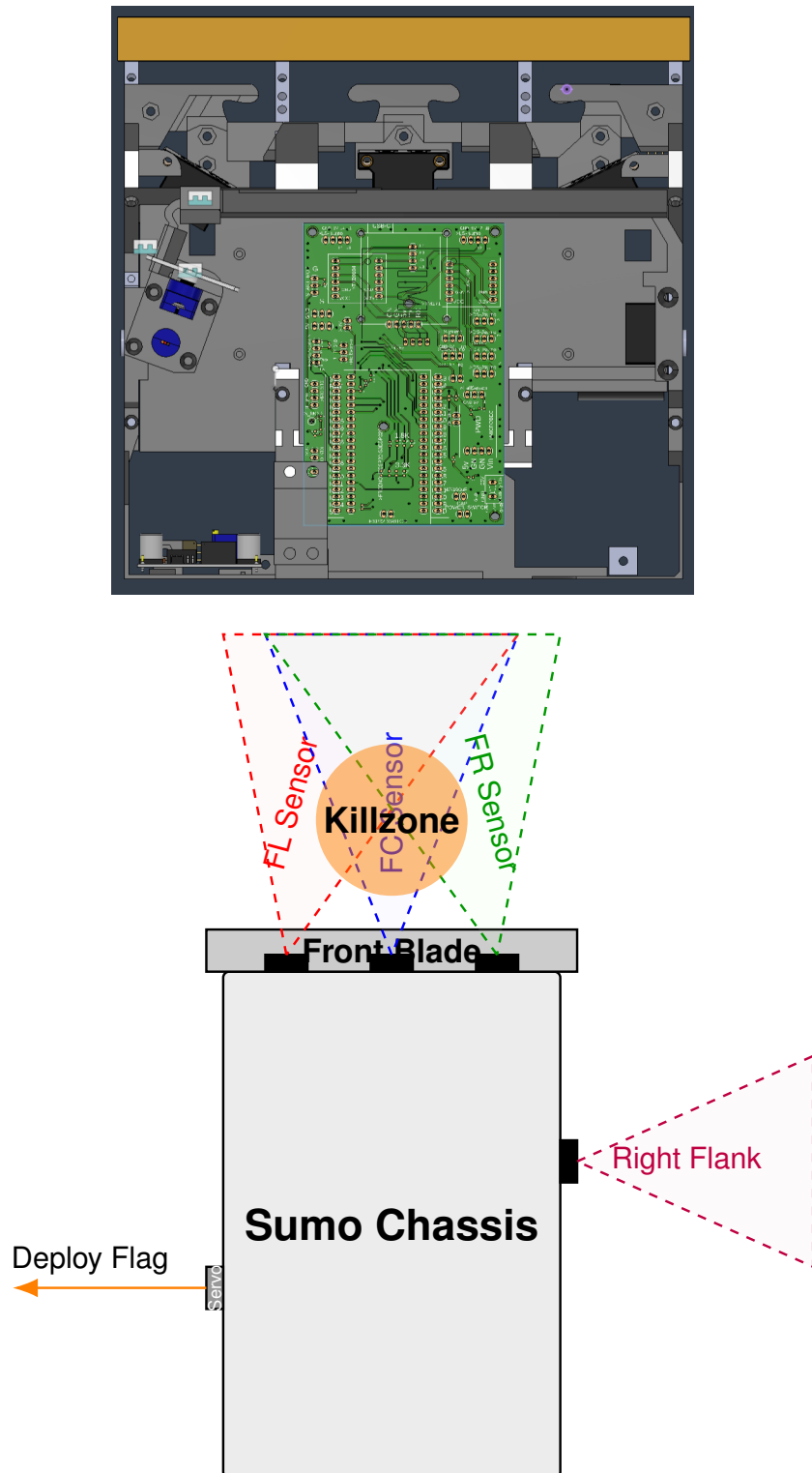


Figure 1: Top-down logic map of the JS200XF sensor focal points and evasive flag deployment.

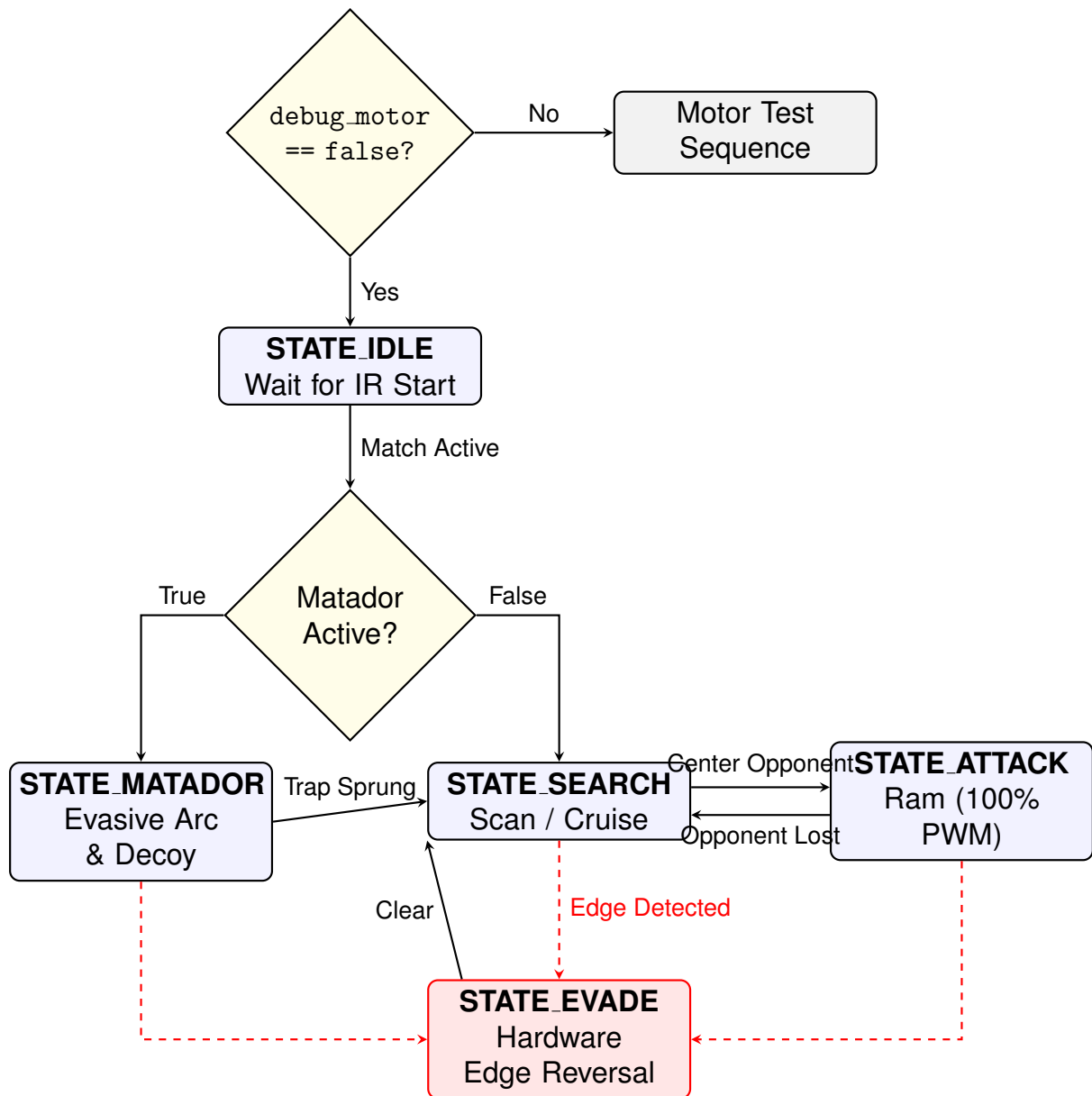


Figure 2: Priority-Based State Machine. Hardware testing is decoupled from the battle strategy, and edge interrupts natively bypass offensive routines.